18-streaming-incremental-aggregates Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all Connect Share Publish

Cmd 1

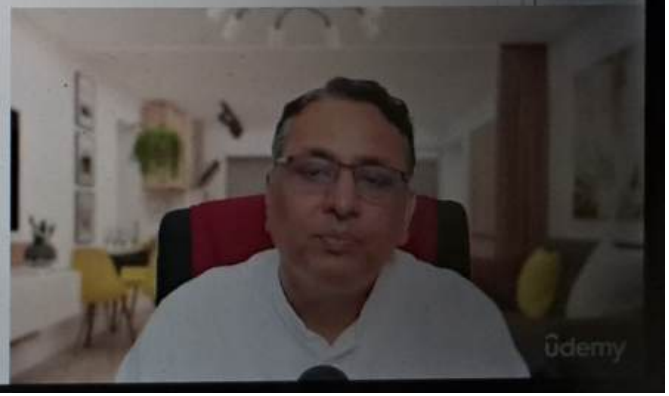
Python

```
class Bronze():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def getSchema(self):
        return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,
                CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,
                PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
                DeliveryType string,
                DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,
                State string>,
                InvoiceLineItems array<struct<ItemCode string, ItemDescription string,
                    ItemPrice double, ItemQty bigint, TotalValue double>>
            """

    def readInvoices(self):
        from pyspark.sql.functions import input_file_name
        return (spark.readStream
                    .format("json")
                    .schema(self.getSchema())
                    .load(f"{self.base_data_dir}/data/invoices")
                    .withColumn("InputFile", input_file_name())
                )

    def process(self):
```



```
                PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
                DeliveryType string,
                DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,
                State string>,
                InvoiceLineItems array<struct<ItemCode string, ItemDescription string,
                    ItemPrice double, ItemQty bigint, TotalValue double>>
            """

    def readInvoices(self):
        from pyspark.sql.functions import input_file_name
        return (spark.readStream
                    .format("json")
                    .schema(self.getSchema())
                    .load(f"{self.base_data_dir}/data/invoices")
                    .withColumn("InputFile", input_file_name())
                )

    def process(self):
        print(f"\nStarting Bronze Stream...", end='')
        invoicesDF = self.readInvoices()
        sQuery =  ( invoicesDF.writeStream
                            .queryName("bronze-ingestion")
                            .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/invoices_bz")
                            .outputMode("append")
                            .toTable("invoices_bz")
                    )
        print("Done")
```

```
class Gold():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def readBronze(self):
        return spark.readStream.table("invoices_bz")

    def getAggregates(self, invoices_df):
        from pyspark.sql.functions import sum, expr
        return (invoices_df.groupBy("CustomerCardNo")
                    .agg(sum("TotalAmount").alias("TotalAmount"),
                         sum(expr("TotalAmount*0.02")).alias("TotalPoints"))
        )

    def saveResults(self, results_df):
        print(f"\nStarting Silver Stream...", end='')
        return (results_df.writeStream
                    .queryName("gold-update")
                    .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/customer_rewards")
                    .outputMode("complete")
                    .toTable("customer_rewards")
                )
        print("Done")

    def process(self):
        invoices_df = self.readBronze()
        aggregate_df = self.getAggregates(invoices_df)
```

```
def upsert(self, rewards_df, batch_id):
    rewards_df.createOrReplaceTempView("customer_rewards_df_temp_view")
    merge_statement = """MERGE INTO customer_rewards t
            USING customer_rewards_df_temp_view s
            ON s.CustomerCardNo == t.CustomerCardNo
            WHEN MATCHED THEN
            UPDATE SET t.TotalAmount = s.TotalAmount, t.TotalPoints = s.TotalPoints
            WHEN NOT MATCHED THEN
            INSERT *
        """
    rewards_df._jdf.sparkSession().sql(merge_statement)

def saveResults(self, results_df):
    print(f"\nStarting Silver Stream...", end='')
    return (results_df.writeStream
                .queryName("gold-update")
                .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/customer_rewards")
                .outputMode("update")
                .foreachBatch(self.upsert)
                .start()
            )
    print("Done")

def process(self):
    invoices_df = self.readBronze()
    aggregate_df = self.getAggregates(invoices_df)
    sQuery = self.saveResults(aggregate_df)
```

```
            INSERT
        """
        rewards_df._jdf.sparkSession().sql(merge_statement)

    def saveResults(self, results_df):
        print(f"\nStarting Silver Stream...", end='')
        return (results_df.writeStream
                    .queryName("gold-update")
                    .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/customer_rewards")
                    .outputMode("update")
                    .foreachBatch(self.upsert)
                    .start()
                )
        print("Done")

    def process(self):
        invoices_df = self.readBronze()
        aggregate_df = self.getAggregates(invoices_df)
        sQuery = self.saveResults(aggregate_df)
        return sQuery
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

19-streaming-inremental-aggregates-test-suite Python

File Edit View Run Help Last edit was 11 minutes ago Provide feedback

Run all Connect Share Publish

Cmd 1

Python

```
%run ./16-streaming-aggregates
```

Command took 0.42 seconds -- by prashantkumarpandey@gmail.com at 11/1/2023, 1:47:47 PM on unknown compute

Cmd 2

Python

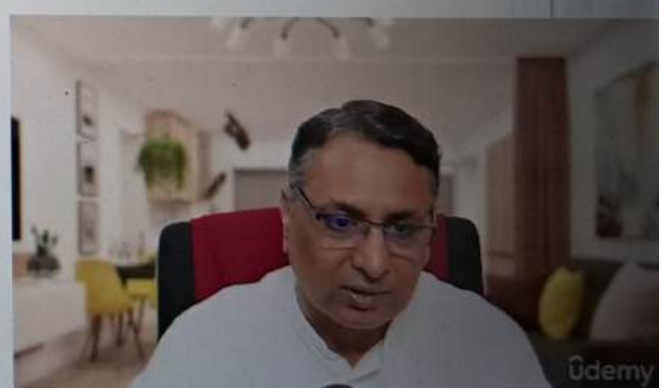
```
class AggregationTestSuite():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def cleanTests(self):
        print(f"Starting Cleanup...", end='')
        spark.sql("drop table if exists invoices_bz")
        spark.sql("drop table if exists customer_rewards")
        dbutils.fs.rm("/user/hive/warehouse/invoices_bz", True)
        dbutils.fs.rm("/user/hive/warehouse/customer_rewards", True)

        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/invoices_bz", True)
        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/customer_rewards", True)

        dbutils.fs.rm(f"{self.base_data_dir}/data/invoices", True)
        dbutils.fs.mkdirs(f"{self.base_data_dir}/data/invoices")
        print("Done")

    def ingestData(self, itr):
```



19-streaming-inremental-aggregates-test-suite Python

File Edit View Run Help Last edit was 11 minutes ago Provide feedback

Run all Connect Share Publish

```
        dbutils.fs.rm("/user/hive/warehouse/customer_rewards", True)

        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/invoices_bz", True)
        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/customer_rewards", True)

        dbutils.fs.rm(f"{self.base_data_dir}/data/invoices", True)
        dbutils.fs.mkdirs(f"{self.base_data_dir}/data/invoices")
        print("Done")

    def ingestData(self, itr):
        print(f"\tStarting Ingestion...", end='')
        dbutils.fs.cp(f"{self.base_data_dir}/datasets/invoices/invoices_{itr}.json", f"{self.base_data_dir}/data/invoices/")
        print("Done")

    def assertBronze(self, expected_count):
        print(f"\tStarting Bronze validation...", end='')
        actual_count = spark.sql("select count(*) from invoices_bz").collect()[0][0]
        assert expected_count == actual_count, f"Test failed! actual count is {actual_count}"
        print("Done")

    def assertGold(self, expected_value):
        print(f"\tStarting Gold validation...", end='')
        actual_value = spark.sql("select TotalAmount from customer_rewards where CustomerCardNo = '226247
        assert expected_value == actual_value, f"Test failed! actual value is {actual_value}"
        print("Done")

    def waitForMicroBatch(self, sleep=30):
        import time
```

```
    def assertGold(self, expected_value):
        print(f"\tStarting Gold validation...", end='')
        actual_value = spark.sql("select TotalAmount from customer_rewards where CustomerCardNo = '2262471989'").collect()[0][0]
        assert expected_value == actual_value, f"Test failed! actual value is {actual_value}"
        print("Done")

    def waitForMicroBatch(self, sleep=60):
        import time
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
        print("Done.")

    def runTests(self):
        self.cleanTests()
        bzStream = Bronze()
        bzQuery = bzStream.process()
        gdStream = Gold()
        gdQuery = gdStream.process()

        print("\nTesting first iteration of invoice stream...")
        self.ingestData(1)
        self.waitForMicroBatch()
        self.assertBronze(501)
        self.assertGold(36859)
        print("Validation passed.\n")
```

```
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
        print("Done.")

    def runTests(self):
        self.cleanTests()
        spark.conf.set("spark.sql.streaming.stateStore.providerClass",
                       "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider")
        bzStream = Bronze()
        bzQuery = bzStream.process()
        gdStream = Gold()
        gdQuery = gdStream.process()

        print("\nTesting first iteration of invoice stream...")
        self.ingestData(1)
        self.waitForMicroBatch()
        self.assertBronze(501)
        self.assertGold(36859)
        print("Validation passed.\n")

        print("\nTesting second iteration of invoice stream...")
        self.ingestData(2)
        self.waitForMicroBatch()
        self.assertBronze(501+500)
        self.assertGold(36859+20740)
        print("Validation passed.\n")
```

19-streaming-inremental-aggregates-test-suite Python

File Edit View Run Help Last edit was 15 minutes ago Provide feedback

Run all Connect Publish

```
        print("\nTesting first iteration of invoice stream...")
        self.ingestData(1)
        self.waitForMicroBatch()
        self.assertBronze(501)
        self.assertGold(36859)
        print("Validation passed.\n")

        print("\nTesting second iteration of invoice stream...")
        self.ingestData(2)
        self.waitForMicroBatch()
        self.assertBronze(501+500)
        self.assertGold(36859+20740)
        print("Validation passed.\n")

        print("\nTesting second iteration of invoice stream...")
        self.ingestData(3)
        self.waitForMicroBatch()
        self.assertBronze(501+500+590)
        self.assertGold(36859+20740+31959)
        print("Validation passed.\n")

        bzQuery.stop()
        gdQuery.stop()
```

Command took 0.09 seconds -- by prashantkumarpandey@gmail.com at 11/1/2023, 1:47:47 PM on unknown compute

Cmd 3

19-streaming-inremental-aggregates-test-suite Python

File Edit View Run Help Last edit was 3 minutes ago Provide feedback

Interrupt my cluster Publish

Python

```
aTS = AggregationTestSuite()
aTS.runTests()
```

Compute snapshot for version: 2

(14) Spark Jobs

```
Starting Cleanup...Done

Starting Bronze Stream...Done

Starting Silver Stream...
Testing first iteration of invoice stream...
        Starting Ingestion...Done
        Waiting for 60 seconds...Done.
        Starting Bronze validation...Done
        Starting Gold validation...Done
Validation passed.


Testing second iteration of invoice stream...
        Starting Ingestion...Done
        Waiting for 60 seconds...
```